Ders 7 Genişletilmiş Notu: Monolitik Cehennemden Kaçış ve Ayrıştırma Stratejileri

Ders 7 Genişletilmiş Notu: Monolitik Cehennemden Kaçış ve Ayrıştırma Stratejileri

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin biçimi
- 2. Bölüm 1: Monolitik mimari
- 3. Bölüm 1: Mikroservis mimarisi
- 4. Bölüm 1: DevOps ve mikroservis
- 5. Bölüm 1: Mikroservis kalıp dili
- 6. Hocanın Bölüm 1 değerlendirmesi: tek hata noktası, bölmek ve orkestrasyon
- 7. Bölüm 2: Yazılım mimarisi nedir?
- 8. 4+1 görünüm modeli
- 9. İşlevsel gereksinimler ve kalite gereksinimleri
- 10. Mimari stiller: katmanlı ve altıgen
- 11. Monolitik mimari ve mikroservis mimarisi (Bölüm 2 bakışı)
- 12. Servis nedir? Komutlar, sorgular ve olaylar
- 13. Gevşek bağlılık ve ortak kütüphaneler
- 14. Servisin boyutu ve dağıtık monolit
- 15. Mikroservis mimarisini tanımlamak: üç adım
- 16. Sistem operasyonlarını belirlemek
- 17. İş yeteneklerine göre ayrıştırma
- 18. Alt alanlara göre ayrıştırma ve hocanın ara özeti
- 19. Ayrıştırma ilkeleri: SRP ve CCP
- 20. Ayrıştırmanın engelleri
- 21. Sistem operasyonlarını servislere atamak ve API tasarımı
- 22. Hocanın kapanış özeti

1. Dersin biçimi

Bu dersten itibaren dersin ikinci yarısı öğrenci sunumlarıyla ilerler. Sunumlar Chris Richardson'ın "Microservices Patterns" kitabının bölümlerini sırayla işler. Bu derste iki sunum yapılır: Bölüm 1 "Escaping Monolithic Hell" (Monolitik cehennemden kaçış) ve Bölüm 2 "Decomposition Strategies" (Ayrıştırma stratejileri). Hoca her sunumdan sonra konuyu özetler, gerektiğinde araya girip düzeltme yapar.

İlk sunum İngilizce yapılmıştır ve ses kaydının büyük kısmı anlaşılmaz durumdadır (Whisper çoğu yerde anlamsız tekrar üretmiştir). Bu yüzden 2.-5. bölümlerin içeriği esas olarak ekrandaki slaytlardan alınmış, altyazıdan seçilebilen cümlelerle tamamlanmıştır.

2. Bölüm 1: Monolitik mimari

2.1 Bölümün hedefleri

Sunumun ilk slaytında bölümde öğrenilecekler sıralanır (ekrandan, 00:05:10):

- Monolitik cehennemin (monolitik mimarinin) ne olduğu, yararları ve zorlukları.
- Mikroservis mimarisinin anlamı, yararları ve sakıncaları.
- Mikroservis mimarisi ile DevOps arasındaki ilişki.
- Mikroservis mimarisi kalıp dilinin ne olduğu ve mimariyi tasarlamak ve iyileştirmek için nasıl kullanıldığı.

2.2 Monolitik mimarinin tanımı

Altyazıdan seçilebildiği kadarıyla sunumda monolitik mimari, çok büyük bir veritabanına ihtiyaç duymayan bir web uygulaması örneğiyle anlatılır. Monolitik mimari yazılım geliştirmenin klasik yoludur: bütün bileşenler tek bir kod olarak geliştirilir, tek parça olarak dağıtılır (deploy) (00:06:16-00:06:41).

Sunumda monolitik mimarinin iki türü anılır: tek süreçli monolit ve modüler monolit (00:08:31). Modüler monolitte tek bir uygulama süreci birden çok modülden oluşur. Bu mimaride veritabanı ortaktır ama modüller birbiriyle haberleşir (00:08:58).

2.3 Yararları ve sorunları

Slayttaki yararlar şunlardır (ekrandan, 00:10:00):

- Geliştirmesi basittir: tek bir uygulama kurmaya odaklanılır.
- Uygulamada köklü değişiklik yapmak kolaydır: kod ve veritabanı şeması değiştirilir, derlenir ve dağıtılır.
- Test etmesi doğrudandır.
- Dağıtması doğrudandır.
- Ölçeklemesi kolaydır.

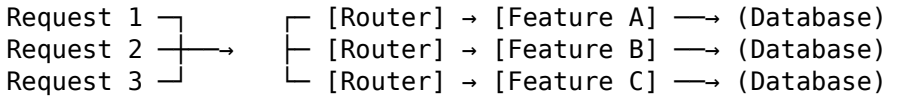
Kısaca monolit, geliştirmesi, test etmesi, dağıtması ve ölçeklemesi kolay bir yapıdır. Aynı slaytta monolitik mimarinin sorunları da verilir:

- **Güvenilirlik:** Veritabanı çökerse ona bağlı bütün özellikler çalışmaz.
- **Güncelleme:** Tek büyük kod tabanı ve sıkı bağlılık yüzünden, her değişiklikte bütün uygulama yeniden dağıtılmak zorundadır.
- **Teknoloji yığını:** Teknoloji değiştirmek hem maliyet hem geliştirme süresi açısından pahalıdır.

3. Bölüm 1: Mikroservis mimarisi

3.1 Tanım

Slayttaki tanım (ekrandan, 00:13:10): mikroservis mimarisi, uygulamanın bir ya da daha çok kod tabanından oluşması ve bunların bağımsız olarak dağıtılabilmesidir. Slayttaki şekil metne dökülürse:



Her özellik kendi yönlendiricisi ve kendi veritabanıyla ayrı bir parça olarak durur. Altyazıdan seçilebildiğine göre mikroservis mimarisi uygulamanın her işlevini ayrı ele alır (00:13:09). Sunumda mikroservislerin Facebook ve Twitter gibi sosyal medya şirketlerinde, Amazon gibi perakendecilerde kullanıldığı söylenir (00:13:50).

3.2 Yararları

Slayttaki yararlar (ekrandan, 00:14:40):

- Büyük ve karmaşık uygulamalarda sürekli teslim (continuous delivery) ve sürekli dağıtımı mümkün kılar.
- Servisler küçüktür ve bakımı kolaydır.
- Servisler bağımsız olarak dağıtılabilir.
- Servisler bağımsız olarak ölçeklenebilir.
- Takımların özerk çalışmasını sağlar.
- Yeni teknolojileri denemeyi ve benimsemeyi kolaylaştırır.
- Hata yalıtımı (fault isolation) daha iyidir.

Sunumun sonunda hata yalıtımı için şu açıklama yapılır: uygulamanın bir yerinde oluşan hata diğer işlevleri etkilemez (00:30:46).

3.3 Sakıncaları

Slayttaki sakıncalar (ekrandan, 00:20:00):

- **Doğru servis kümesini bulmak zordur.** Daha çok servis daha çok kaynak demektir; bu da veritabanı ve transaction yönetimini zorlaştırır.
- **Dağıtık sistemler karmaşıktır.** Geliştirme, test ve dağıtım zorlaşır. Mikroservislerde test yapılmadan önce her bağımlı servisin doğrulanması gerekir.
- **Birden çok servise yayılan özelliklerin dağıtımı dikkatli koordinasyon ister.** Ürünün birden çok servis arasında düzenlenmesi gerekebilir ve bu monolitteki kadar doğrudan değildir.
- **Mikroservis mimarisine ne zaman geçileceğine karar vermek zordur.** Mikroservis tasarımının karmaşıklığı mimariyi, organizasyonu ve geliştirme sürecini değiştirir.

Altyazıdan seçilebildiğine göre sunumda her servisin CPU, bellek ve giriş-çıkış kaynaklarının ayrıca izlenmesi gerektiği de anılır (00:20:18).

3.4 Ne zaman mikroservis?

Sunumda basit bir ölçüt verilir: küçük bir veritabanı ve küçük bir uygulama için mikroservise gerek yoktur, monolitik mimari yeterlidir; büyük bir sistemde ise mikroservis gerekir (00:25:25-00:25:37).

4. Bölüm 1: DevOps ve mikroservis

DevOps (development operations), ürün yönetimi, yazılım geliştirme ve operasyon uzmanları arasındaki iletişimi ve iş birliğini güçlendiren bir yazılım geliştirme sürecidir (00:21:15). Slayttaki maddeler (ekrandan, 00:21:30):

- DevOps, geliştiricilerin ve BT operasyon ekiplerinin daha yakın çalışarak daha kaliteli yazılımı daha hızlı teslim etmesini sağlayan bir yöntemdir.
- Geliştiricilere üretim ortamını görme imkânı vererek yazılım kalitesini artırmalarına yardım eder.
- Yazılımı daha küçük parçalara bölerek teslim hızını ve kaliteyi artırır.
- Mikroservisler DevOps takımlarının bağımsız işlev parçalarını paralel geliştirmesini sağlar. Bunun sonuçları: yeniliği hızlandırmak, DevOps takımlarını daha üretken yapmak, hataları azaltmak, yazılım geliştirme maliyetini azaltmak, ürün kalitesini artırmak ve daha yüksek iş değeri üretmek.

Slaytın sonucu: başka bir deyişle DevOps, mikroservis mimarisinin sakıncalarını azaltır. Sunumda DevOps ile mikroservisin birlikte çalışarak yazılım geliştirmenin hızını artırdığı, DevOps'un yazılım yinelemesi sürecini otomatikleştirip izlediği söylenir (00:21:39-00:21:49).

5. Bölüm 1: Mikroservis kalıp dili

Slayttaki tanımlar (ekrandan, 00:25:30):

- Kalıp (pattern) ve kalıp dili (pattern language) kavramları Christopher Alexander tarafından 1977'de ortaya konmuştur.
- **Kalıp dili:** belirli bir alandaki problemleri çözen ilişkili kalıplar topluluğudur.

- **Kalıp:** belirli bir bağlamda ortaya çıkan bir probleme yeniden kullanılabilir çözümdür.
- **Mikroservis mimarisi kalıp dili:** geliştiricinin bir uygulamayı mikroservis mimarisiyle kurmasına yardım eden kalıplar topluluğudur.

Kalıp dilinin iki ana amacı vardır:

1. Mikroservislerin uygulama için uygun olup olmadığına karar vermeyi sağlamak.
2. Mikroservis mimarisini başarıyla kullanmayı sağlamak.

Sunumda mikroservise geçildiğinde bu kararın sonuçlarıyla başa çıkmak için pek çok kalıp seçmek gerektiği söylenir; ilk karar da uygulamanın servislere nasıl ayrıştırılacağıdır (00:27:00-00:27:10). Kalıplar arasındaki ilişkiler de anılır (00:28:34-00:28:46):

- **Öncül (predecessor):** Bu kalıba ihtiyaç doğuran kalıp.
- **Ardıl (successor):** Bu kalıbın ortaya çıkardığı bir sorunu çözen kalıp.
- **Alternatif:** Aynı probleme başka bir çözüm sunan kalıp.

  Ek bilgi (derste anlatılmadı): Richardson'ın kitabında örneğin mikroservis mimarisi kalıbı, monolitik mimari kalıbının alternatifidir; mikroservis seçildiğinde ortaya çıkan servis keşfi, veri tutarlılığı gibi sorunları çözen kalıplar da onun ardıllarıdır. Kitap bu ilişkileri büyük olasılıkla bir kalıp dili haritasıyla gösterir.

6. Hocanın Bölüm 1 değerlendirmesi: tek hata noktası, bölmek ve orkestrasyon

Hoca sunumu şöyle özetler (00:31:46). Geçen hafta görülen istemci-sunucu mimarisi aslında monolitik bir yapı oluşturur. Monolitik yapı güzeldir, çünkü her şey bir yerde, tek parça durur. Bu yüzden hızlı yapılabilir, bir yere koyulup çalıştırılabilir.

Sorun şuradadır: en ufak bir değişiklik yapılmak istendiğinde her şeyi baştan dağıtmak gerekir, çünkü ortada kocaman bir uygulama vardır. Ayrıca her şey tek yerdeyse klasik tek hata noktası (single point of failure) problemine düşülür: bir şey patladığı anda her şey patlar. Hoca, devlet dairelerinde sık duyulan "sistem çöktü" sorunlarının büyük çoğunluğunun bu eski teknolojiyle ya da monolitik mimariyle geliştirilmiş uygulamalardan kaynaklandığını söyler. Örneğin veritabanı sunucusunun ağı koptuğunda her şey çöker, çünkü tek bir veritabanı sunucusu vardır ve her şey oradadır. Bir tablo bozulduğunda da bütün uygulama çöker; sistem sorguları düzgün yapamadığı için hata ya da tepki vermeme moduna geçer.

İlk akla gelen çözüm, madem her şey tek yerde, bunları bölmektir. Hoca bunu bir benzetmeyle anlatır: bir tabak yemeği tek seferde ağza atıp yiyemeyiz, ama çatal bıçakla küçük lokmalara bölersek rahatça çiğneyip yutabiliriz. Nasıl bölünecek? İşlev bazında: derste hep tepki veren sistemler geliştirmeye çalışıldığı söylendi; kullanıcı tepkileri ya da sistemin hizmetleri birer servis olarak modellenir. Hatta bu servisler, parçalar birbirinden olabildiğince bağımsız çalışabilsin ve geliştirilebilsin diye bölünebildiği kadar bölünür.

Ama bölündükten sonra bunun bir de yönetimi vardır. Hangi servis çağrılacak, hangi servis neyi hangi sırayla çağıracak, bir işlem geldiğinde hangi servis önce çalışacak, hangisi başka bir iş yaparken diğerini bekleyecek, bir kuyruk mekanizması kurulacak mı, hangi veriye hangi veritabanı parçasından nasıl erişilecek, bunlar için hizmet veren mikroservisler nasıl keşfedilecek? Bir sürü yeni problem çıkar; bunlara **orkestrasyon** problemleri denir. Ne kadar bölünürse parçacıkları yapmak o kadar kolaylaşır, ama yönetimde sıkıntılar başlar. Bunun için çeşitli kalıplar (tasarım kalıpları) vardır.

Sunumda anılan Christopher Alexander bilgisi için hoca, bu ders kapsamında çok fazla kalıp aktarmadığını, asıl derdinin genel olarak bir tasarımın nasıl yapılacağını anlatmak olduğunu söyler (00:35:14). İyi tasarım iyi kalıplar kullanılarak yapılır, çünkü herkes benzer alanlarda benzer problemler yaşar; biri bunun için iyi bir çözüm geliştirir ve bu çözüm benzer problemleri çözmek için de iyi bir yol olur. Bu çözümlere kalıp denir. Mikroservis için de bahsedilen problemleri çözmek üzere ortaya atılmış kalıplar vardır. Dersin geri kalanında bölüm bölüm bu problemler, önerilen çözüm yaklaşımları ve bunların getirdikleri incelenecektir.

7. Bölüm 2: Yazılım mimarisi nedir?

İkinci sunum ayrıştırma stratejilerini, yani bir uygulamanın nasıl bölüneceğini anlatır. Akış şöyledir: yazılım mimarisi ve mimari stiller nedir, bir mikroservis uygulaması nasıl tanımlanır, ilkeleri ve zorlukları nelerdir, bir servis tanımlanırken API'ler neye göre açılır (00:37:28).

Yazılım mimarisi çok boyutlu bir yapıdır: ana parçalardan ve bu parçalar arasındaki bağımlılıklardan oluşur. Temel olarak yazılım öğelerini ve aralarındaki ilişkileri belirler. Önemi, uygulamanın kalite özelliklerini belirlemesindendir (00:38:03-00:38:28). Hoca burada araya girip derste verilen tanımı hatırlatır: en basit tanımıyla bileşenler (components), bağlayıcılar (connectors) ve düzenleşim (configuration). Kitaptaki "yazılım öğeleri, ilişkiler ve özellikler" tanımı da aynı tanımın farklı kelimelerle söylenişidir; mimari her yerde böyle evrilip çevrilip tanımlanır (00:38:49).

Slayttaki ifade (ekrandan, 00:40:00): geleneksel olarak mimarinin hedefi ölçeklenebilirlik, güvenilirlik ve güvenlikti. Bugün mimarinin yazılımın **hızlı** ve **güvenli** teslimini de sağlaması önemlidir. Mikroservis mimarisi, uygulamaya **yüksek sürdürülebilirlik** (maintainability), **test edilebilirlik** ve **dağıtılabilirlik** (deployability) kazandıran bir mimari stildir. Mikroservis mimarisinde uygulama bir **servisler kümesidir**.

Sunucu bu ihtiyacı şöyle açıklar: servislerden biri ya da programın bir kısmı çöktüğünde diğer kısmın bazı durumlarda çalışmaya devam etmesi beklenir. Ayrıca bir uygulamada değişiklik yapıldığında bu değişikliğin canlıya alınması bir hafta sürüyorsa, bu bazı noktalarda gerçekten kritik zafiyetlere yol açabilir (00:40:16).

8. 4+1 görünüm modeli

Slayttaki içerik (ekrandan, 00:41:40): Perry ve Wolf'a göre yazılım mimarisi = {Elemanlar, Biçimler, Gerekçe/Kısıtlar}. Bir yazılım mimarisini tanımlamak için birden çok görünümden ya da bakış açısından oluşan bir model kullanılır:

- **Mantıksal görünüm (logical view):** Tasarımın nesne modeli (nesneye yönelik yöntem kullanıldığında). Richardson'a göre programdaki sınıf ve paket yapısıdır.
- **Süreç görünümü (process view):** Tasarımın eş zamanlılık ve senkronizasyon yönlerini yakalar.
- **Fiziksel görünüm (physical view):** Yazılımın donanıma eşlenmesidir ve dağıtık yönünü yansıtır. Sınıf ve paket yapısından tamamen ayrıdır; sistem mühendislerinin, topolojinin ve haberleşmenin alanıdır.
- **Geliştirme görünümü (development view):** Yazılımın geliştirme ortamındaki statik düzenidir; uygulamanın modüler yapısını gösterir.
- **+1 görünüm:** Kullanım durumları ya da senaryolar beşinci bir görünüm olabilir (zorunlu değil).

Sunucu modelin Philippe Kruchten'e ait olduğunu söyler ve Richardson'ın anlatımının daha çok Java ve Java benzeri uygulamalar üzerinden olduğunu, başka dillerle tam oturmayabileceğini belirtir (00:42:03). Richardson'a göre elimizdeki kullanım durumları ve senaryolar ayrı bir boyut olduğu için beşinci katman olarak eklenebilir. Mantıksal görünüm doğrudan geliştirme ve süreç görünümleriyle ilişkilidir (00:43:14).

> **Ders sonrası not:** Slaytta ve sunumda +1 görünümün adı "animation view" olarak geçer. Kruchten'in modelinde bu görünüm büyük olasılıkla "scenarios" (senaryolar) görünümüdür; Richardson kitabında senaryoların diğer dört görünümü "canlandırdığını" (animate) söyler, "animation" adı buradan gelen bir karışıklık olabilir.

9. İşlevsel gereksinimler ve kalite gereksinimleri

Kullanılan mimari, uygulamayı iyileştiren ya da kötüleştiren özellikler getirir; bunlar kalite özelliklerini belirler (00:44:40). Gereksinimler (isterler) iki kategoride sıralanır (00:45:02):

- **İşlevsel gereksinimler:** Programın ne iş yapacağıdır. Genellikle kullanım durumları ve kullanıcı hikâyeleri biçiminde gelir.
- **Kalite gereksinimleri:** Sistemin ölçeklenebilirliği, dayanıklılığı gibi müşteri tarafından istenen nitelikler.

Örnek (00:45:41): bir sistemde kullanıcının giriş yapabilmesi işlevsel bir gereksinimdir. Ama "kullanıcı 10 saniyede giriş yapabilsin" denirse bu bir kalite gereksinimidir.

10. Mimari stiller: katmanlı ve altıgen

10.1 Katmanlı mimari

Slayttaki tanım (ekrandan, 00:46:20): katmanlı mimari, bir mimari stilin klasik örneğidir; yazılım öğelerini katmanlar hâlinde düzenler. Popüler üç katmanlı (three-tier) mimari, katmanlı mimarinin mantıksal görünüme uygulanmasıdır:

- **Sunum katmanı (presentation layer):** Kullanıcı arayüzünü ya da dış API'leri gerçekleştiren kod.
- **İş mantığı katmanı (business logic layer):** İş mantığı.
- **Kalıcılık katmanı (persistence layer):** Veritabanıyla etkileşim mantığı.

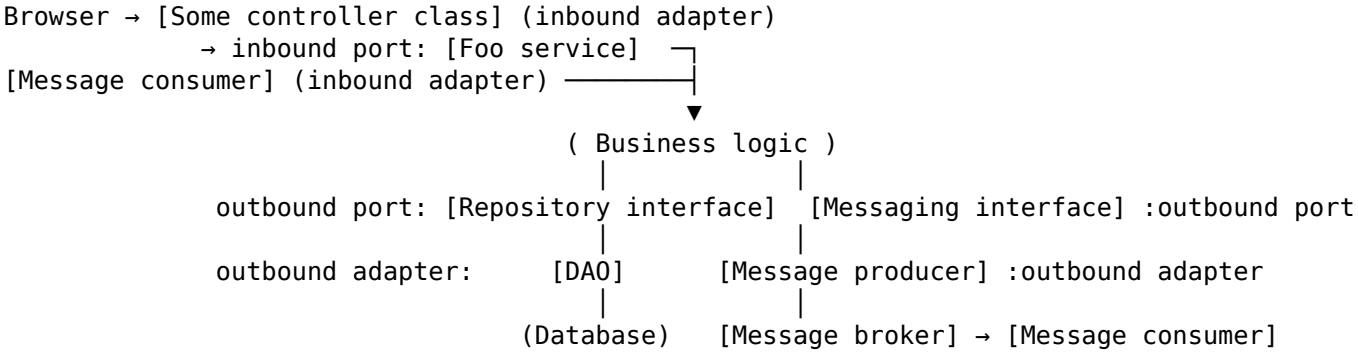
Sunum katmanı iş mantığıyla, iş mantığı da kalıcılık katmanıyla ilişkilidir. Klasik ve en popüler yöntemlerden biridir. Sunumda anılan sakıncalar (00:46:38-00:47:13): tek bir sunum katmanı olduğu için çok fazla entegrasyon yapılamaz, yapı tek bir sisteme göre kalır; tek bir kalıcılık katmanı olduğu için birden fazla veritabanına bağlanmak zorlaşır ve büyük sistemler tanımlanırken sıkıntı çıkar.

10.2 Altıgen (hexagonal) mimari

Belirli girdiler alınan durumlarda iş mantığı katmanı sıkıntıya düştüğü için altıgen mimari önerilmiştir (00:47:29). Slayttaki içerik (ekrandan, 00:48:10):

- Katmanlı mimari stiline bir alternatiftir.
- Mantıksal görünümü **iş mantığını merkeze koyacak** biçimde düzenler.
- Sunum katmanı yerine, dışarıdan gelen istekleri iş mantığını çağırarak karşılayan bir ya da daha çok **gelen adaptör (inbound adapter)** vardır.
- Kalıcılık katmanı yerine, iş mantığının çağırdığı ve dış uygulamaları çağıran bir ya da daha çok **giden adaptör (outbound adapter)** vardır.
- İş mantığının bir ya da daha çok **portu** vardır. Port bir işlemler kümesi tanımlar ve iş mantığının dışarısıyla nasıl etkileştiğini belirler.

Slayttaki şekil metne dökülürse:



Sunucunun açıklamasıyla: iş mantığı sistemin çekirdeğidir (kernel), altıgen de uygulamanın kendisidir. Uygulamaya dışarıdan girilirken ya da çıktı verilirken adaptörler kullanılır: giriş için gelen, çıkış için giden adaptörler. İş mantığı önemlidir, kolay değiştirilmemeli ve dışarıdan çok fazla etkilenmemelidir; bu yüzden yalnız uygulamayla haberleşir: girdi alıyorsa gelen port, çıktı veriyorsa giden port üzerinden (00:48:31-00:48:38).

Katmanlı yapıdan farklı olarak veritabanıyla iletişim için veri erişim nesneleri (DAO, Data Access Object) kullanılır. DAO, birden fazla veritabanıyla haberleşmeyi sağlar; ileride veritabanı sistemi değiştirilecekse arkada bir arayüz oluşturulduğu için bu değişiklik çok fazla kod yazmadan, maliyetsiz yapılabilir (00:49:05-00:49:32).

11. Monolitik mimari ve mikroservis mimarisi (Bölüm 2 bakışı)

11.1 Monolitik mimari

Slayttaki içerik (ekrandan, 00:50:00): monolitik mimari, gerçekleme görünümünü tek bir bileşen olarak, yani **tek bir çalıştırılabilir dosya ya da WAR dosyası** olarak yapılandıran bir mimari stildir. Uygulama ya da takım büyüdükçe bu yaklaşımın sakıncaları giderek belirginleşir.

- **Yararları:** Geliştirmesi, dağıtması, ölçeklemesi ve test etmesi basittir.
- **Sakıncaları:** Büyük monolitik kod; aşırı yüklenmiş web kapsayıcısı (web container); sürekli dağıtımın zorluğu; geliştirmeyi ölçeklemenin zorluğu; bir teknoloji yığınına uzun süreli bağlılık gereği.

Slayttaki "geleneksel web uygulaması mimarisi" şekli:

```
[Browser] ⇄ [Apache] ⇄ [ Tomcat
                           └─ WAR: StoreFrontUI, Accounting Service,
                                   InventoryService, Shipping Service ] ⇄ [MySQL Database]
```

Sunucu, takım büyüdükçe kodun biriktiğini ve geliştirme süresinin arttığını anlatır. Tomcat üzerinde çalışan bazı uygulamaların 200-300 MB'tan başlayıp büyüdüğünü, sınıf yapılarının birbirine girdiğini, test ve yeni geliştirmede sıkıntı çıktığını söyler; yine de basit uygulamalar için tercih edilebilir bir yöntemdir (00:50:23-00:50:49). Teknolojiyi değiştirmek, örneğin Tomcat'ten başka bir uygulama sunucusuna geçmek ya da Apache'den kurtulmak isteniyorsa monolit uygun değildir. Sunucunun yorumuna göre kullanılan teknolojinin uzun süre kullanılmasını gerektirdiği için kamu ve askeri sistemlere daha uygundur (00:51:06).

11.2 Mikroservis mimarisi

Mikroservis daha uyarlanabilir, hata payı düşük bir sistemdir ve yeni teknolojiler kullanılabilir (00:51:30). Gereksinimler birden çok API üzerinden sunulur; arka planda servislerin verdiği işlevlerle sistem çok daha basit hâle gelir, karmaşık uygulamalar daha basit parçalarla çözülür. Monolitte tek bir WAR tek bir sunucuda çalışırken, burada birden çok çalıştırılabilir dosya ya da WAR olabilir. Temel parçalar servisler, bağlayıcılar ve iletişim protokolleridir (00:51:43-00:52:18).

Yararları:

- Her parça (servis) ayrı bir gruba verilir. Servisler büyük bir sistemin küçük parçaları olduğu için test edilebilirlik ve sürdürülebilirlik artar.
- Yalnız küçük bir parça değiştiği için canlıya alınabilirlik artar ve geliştirme eforu azalır.
- Servisler arasında yalıtım olduğu için geliştirici yalnız kendi kısmını ve diğer servislerin sunduğu API'leri bilmekle yetinir; bütün sistemi bilmek zorunda kalmaz.
- Monolitte bütün işlevler tek bir dosyadaydı; burada her özellik ayrı paketlerdir.
- Hata yalıtımı: bir servisteki hata bütün sistemi etkilemez. Ama birbirine bağlı servisler zincirleme bir işlem yapıyorsa yine sorun olabilir; bu da belli tasarım biçimleriyle aşılır.
- Bir servisin teknolojisi değiştirilerek o servis yenilenebilir; efor yalnız o servisin geliştirme ve test aşamasıdır.

Sakıncaları (00:54:34-00:56:47):

- Birden çok geliştirici çalıştığında sistem karışabilir. Bir servis başka birinin servisine bağlıysa ve o servis değişirse sorun çıkar; merkezi bir kontrol ya da servis geliştiricileri arasında iletişimle çözülür.
- Servisler arası haberleşme yüzünden test zorlaşır; diğer servisin tam olarak ne yaptığı bilinmeyebilir. Dokümantasyon ve iletişimle aşılır; burada sıkıntı insan faktörüdür.
- Sistem büyüdüğü için dağıtım zorlaşır ve her servis ayrı bir program olarak başladığı için sistem kaynaklarından daha fazla tüketilir. Örnek: Java ile yazılmış üç servis varsa, servisler ne kadar küçük olursa olsun her biri için bir Java sanal makinesi ayağa kaldırılır ve bu aşırı bellek kullanımına yol açabilir. Sunucunun yorumuna göre servisler dağıtık çalıştırıldığı için bu daha az önemli bir sorundur.

12. Servis nedir? Komutlar, sorgular ve olaylar

Slayttaki içerik (ekrandan, 00:57:30):

- Servis, işe yarar bir işlevi gerçekleyen, kendi başına duran ve bağımsız dağıtılabilen bir yazılım bileşenidir.
- Servisin, istemcilerinin işlevlerine erişmesini sağlayan bir API'si vardır.
- İki tür işlem vardır: **komutlar** ve **sorgular**. createOrder() gibi bir komut işlem yapar ve veriyi günceller. findOrderById() gibi bir sorgu veriyi getirir.
- Servis ayrıca OrderCreated gibi, istemcilerinin tükettiği **olaylar** yayımlar.
- API; komutlar, sorgular ve olaylardan oluşur.
- Mikroservis mimarisindeki her servisin kendi mimarisi ve olası olarak kendi teknoloji yığını vardır. Ama tipik bir servis altıgen mimariye sahiptir.

Slayttaki şekil metne dökülürse:

```
[Order Service client] ──invokes──→ Service API (işlemleri tanımlar)
                                     ├─ Commands: createOrder() ...
                                     └─ Queries:  findOrderById() ...
[Order Service client] ←─subscribes── [Order event publisher]
                                       (Order created, Order cancelled;
                                        veri değişince olay yayımlar)
                                     → hepsi altıgen [Order Service] üzerinde
```

Sunucu, CRUD (create, read, update, delete) işlemlerinden okuma dışındakilerin, yani veritabanındaki veriyi değiştirebilecek işlemlerin komut, okuma işlemlerinin sorgu olduğunu söyler (00:57:17). Restoran sistemi örneği (00:57:44): bir sipariş oluşturulurken veritabanına kayıt atıldığı için "sipariş oluştur" (create order) bir komuttur. "Siparişim ne durumdaydı?" diye sorgulama yapıldığında yalnız okuma olduğu için bu bir sorgudur.

Servisler temel olarak altıgen mimariyle tasarlanır; ama her mimarinin kendi gereksinimleri olduğu için bazen altıgen mimarinin dışına da çıkılabilir (00:58:21).

13. Gevşek bağlılık ve ortak kütüphaneler

13.1 Gevşek bağlılık (loose coupling)

Slayttaki içerik (ekrandan, 01:00:00): mikroservis mimarisinin önemli bir özelliği servislerin gevşek bağlı olmasıdır. Gevşek bağlı servisler, uygulamanın geliştirme zamanı özelliklerini, özellikle **sürdürülebilirlik** ve **test edilebilirliği** iyileştirmenin anahtarıdır. Bu servisleri **anlamak**, **değiştirmek** ve **test etmek** çok daha kolaydır.

Yararları:

- Daha az karşılıklı bağımlılık.
- Daha az koordinasyon.
- Daha az bilgi akışı.
- Daha esnek yapı.

Slayttaki şekilde iki çizge karşılaştırılır: solda dört düğümün her biri diğerleriyle çift yönlü oklarla bağlıdır ("Tightly Coupled - Many Dependencies"); sağda altı düğüm arasında yalnız birkaç ok vardır ("Loosely Coupled - Some Dependencies").

Sunucunun açıklaması (00:59:11): her daire bir servis olsun. A servisi B servisiyle ilgili bir işlem yaparken B'ye muhtaçsa bu, mikroservis kullanımında büyük sıkıntılara yol açar. Richardson'ın örneğinde sipariş servisi başka bir servisi, o da başka bir servisi bekliyorsa sistem çalışmayabilir ya da geç cevap verebilir. Bu yüzden servisler arasındaki ilişki ve bağımlılık olabildiğince az tutulur; gevşek bağlılık sıkı bağlılığın (tightly coupled) tersidir. Bağlantılar ve gereksinimler yine olacaktır ama olabildiğince düşük tutulur. Böylece diğer modüllere ihtiyaç kalkar, kodu değiştirmek ve test etmek kolaylaşır.

Sunucunun yorumuyla (01:00:54-01:01:38): servisler arasında koordinasyon gerekmediği için senkronizasyondan kaynaklanan iletişim problemi büyük ölçüde ortadan kalkar; bağlantılar az olduğu için ihtiyaçlar azalır; bilgi akışı az olduğu için ağ rahatlar; bir servisin çökmesi diğerini etkilemez. Bu yapı sıkı bağlı mimariye göre daha düzenlenebilir ve esnektir.

13.2 Ortak (shared) kütüphaneler

Birden fazla servisin ortak kullandığı kütüphaneler olabilir: üçüncü taraf yazılımlar ya da ekibin kendi yazdığı kütüphaneler (01:01:41). Bu kütüphaneler zamanla değişebilir ve bu bir sorun kaynağıdır. Öte yandan servisler aynı ya da çok benzer işler yapıyorsa her biri için kütüphaneyi ayrı ayrı tutmak toplamda büyük bir yük getirir: her biri 4-5 MB, bazen çok daha fazla olabilir; sistem karmaşıklaştıkça aynı kütüphane 10, 20, bazen 30 kez ayrı ayrı tanımlanır (01:02:10-01:02:43). Ortak kullanım disk kullanımını azaltır ve kod tekrarını önler.

Ama bazı mimarilerde ortak kütüphane kullanılamaz. Sunucunun örneği (01:03:00): bir kütüphanenin 1.5 sürümünde A uygulaması kararlı çalışıyor olsun; kütüphane başka bir uygulamanın ihtiyacına göre değiştirildiğinde A'nın işlevselliği azalabilir. Bu durumda her uygulamanın kendi sürümünü kullandığı statik kütüphane gerekir. Sunucuya göre ortak kütüphane, akışları birbirine benzeyen uygulamalar için kullanılır; uygulamaların birbirine bağımlılığını ortadan kaldırmak için ise statik kütüphane tercih edilir. Ortak kütüphaneyi hangi uygulamaların hangi özellikleriyle kullandığı bilindiği ölçüde onu değiştirme hakkı sınırlanır; aksi hâlde geliştirme maliyeti çok artar (01:04:07).

> **Ek bilgi (derste anlatılmadı):** Richardson'ın kitabında ortak kütüphaneler için önerilen ölçüt büyük olasılıkla şudur: değişmesi beklenmeyen işlevler (ör. genel bir Money sınıfı) kütüphaneye konabilir; ama değişmesi muhtemel bir iş mantığı kütüphanede tutulursa her değişiklikte onu kullanan bütün servisler yeniden derlenip dağıtılmak zorunda kalır, bu durumda o işlevi ayrı bir servis yapmak daha iyidir.

14. Servisin boyutu ve dağıtık monolit

Sunucu, mikroservisin monolitten kat kat küçük olduğunu, ama "mikro" sözcüğünün çok çok küçük olmayı gerektirmediğini söyler; önemli olan işlevlerin ayrılması ve kaynak tüketiminin olabildiğince azaltılmasıdır (01:04:38). Çok küçük parçalara, hatta fonksiyon bazında bölmenin sıkıntısı vardır: çok büyük takımlar gerekir ve geliştirme maliyeti artar. Sunucunun benzetmesi (01:05:33): bir binanın bütün olarak teftiş edilmesi ile her katının tek tek teftiş edilmesi arasındaki fark gibi; toplam harcanan zaman artar.

Hoca burada araya girip kitabın söylemek istediğini düzeltir (01:06:12):

> **Hocanın vurgusu:** Mikroservisin küçük olması istenir ki adına uygun olsun, ama aslında boyut çok önemli değildir. Yeter ki dağıtık bir monolitik mimari (distributed monolithic architecture) yapılmasın. Bir değişiklik birden çok servisin birlikte değişmesini gerektiriyorsa servis kırılımında sorun vardır ve mikroservis tasarımı gözden geçirilmelidir, çünkü yapılan şey mikroservis değildir; monolitik mimari dağıtık hâle getirilmiş olur.

Hoca, günümüzde pek çok insanın ya da kurumun tam olarak bunu yaptığını söyler: "mikroservis yapıyoruz" derler ama yaptıklarının büyük çoğu monolitik mimariyi mikroservislere dağıtmaktır. Mantığın farklı olması gerekir (01:06:54-01:07:08).

15. Mikroservis mimarisini tanımlamak: üç adım

Sunucu bu kısmı üç başlıkta toplar: sistem operasyonlarının tanımlanması, iş yeteneğinin (business capability) ne olduğu ve nasıl tanımlandığı, alan güdümlü tasarım (Domain-Driven Design) ve alt alan (subdomain) kalıbı (01:08:13-01:08:30).

Bilgisayar biliminde genellikle bir problem ve onu çözen bir algoritma vardır. Ama sistem tasarımı için böyle genel işlem basamakları geliştirilmiş değildir; benzer sistemler benzer adımlarla tasarlanınca çözüm bulunabilir (01:08:54-01:09:17). Slayttaki ifadeler (ekrandan, 01:10:00):

- Bir mikroservis mimarisini nasıl tanımlamalıyız?
- Bütün sistem tasarımı problemlerini çözen sabit bir süreç adımı var mı?
- Bir uygulamanın mimarisini tanımlamak **mekanik olarak izlenebilecek bir süreç değildir**.
- **Mimariyi tanımlamak bilimden çok sanattır.**

Slayttaki şekil, kitaptaki üç adımlı süreci gösterir:

```
Başlangıç: gereksinimler (kullanıcı hikâyeleri)
  "As a consumer I want to place an order so that I can ..."
  "As a restaurant I want to accept an order so that I can ..."

Adım 1: Sistem operasyonlarını belirle
  (bir sistem operasyonu bir dış isteği temsil eder)
  FTGO ← createOrder(), acceptOrder()

Adım 2: Servisleri belirle
  FTGO = { Order Service, Restaurant Service, Kitchen Service, ... }

Adım 3: Servis API'lerini ve iş birliklerini tanımla (yinelemeli)
  createOrder() → Order Service —verifyOrder()→ Restaurant Service
                  Order Service —createTicket()→ Kitchen Service
  acceptOrder() → Kitchen Service
```

Sunucunun anlatımıyla (01:10:00): önce sistem gereksinimleri, sonra arka planda gereken servisler, sonra bu servislerin dışarıya nasıl açılacağı, yani API'ler tasarlanır. Başta elde yalnız hikâyeler ve senaryolar vardır; bunların birleştirilip temel bir yapıya getirilmesi gerekir. Bunun sonucu, sistem ayrıntılarını vermeyen ama aralarındaki ilişkiyi belirten yüksek seviyeli alan modelidir (high-level domain model) (01:10:15-01:10:43).

> **Hocanın vurgusu:** Bu, derste zaten anlatılan şeydir: UML sınıf diyagramıyla önce bir alan modeli, yani analiz modeli çıkarılır. Slayttaki resim tam olarak bunu gösterir (01:10:55).

Sunucu, kitabın hikâye ve senaryoların oluşturulması aşamasını atlayıp sanki bunlar hazır gelmiş gibi, yani müşteri gelip "böyle bir ihtiyacımız var" demiş gibi ilerlediğini ekler (01:11:07).

16. Sistem operasyonlarını belirlemek

Slayttaki içerik (ekrandan, 01:12:45): yüksek seviyeli alan modelini tanımladıktan sonraki adım, uygulamanın karşılaması gereken istekleri belirlemektir. İki tür sistem operasyonu vardır:

- **Komutlar:** Veri **oluşturan**, **güncelleyen** ve **silen** sistem operasyonları.
- **Sorgular:** Veri **okuyan** (sorgulayan) sistem operasyonları.

Bu sistem operasyonları isteklere karşılık gelir. Bu aşamada işlemleri olabildiğince soyut düşünmek yararlıdır.

Sunucu ayrımın nedenini şöyle açıklar (01:11:57): okuma işlemi sistemin çalışabilirliğini doğrudan etkilemez. Güvenilirlik açısından okuma da sorun olabilir ama yeni bir şey oluşturma, güncelleme ya da silme kadar kritik değildir.

Slayttaki örnek komutlar tablosu:

Aktör	Hikâye	Komut	Açıklama
Consumer	Create Order	`createOrder()`	Bir sipariş oluşturur.
Restaurant	Accept Order	`acceptOrder()`	Restoranın siparişi kabul ettiğini ve belirtilen zamanda hazırlamayı üstlendiğini gösterir.

Slayttaki createOrder() komutunun belirtimi:

```
Operation:       createOrder(consumer id, payment method, delivery address,
                             delivery time, restaurant id, order line items)
Returns:         orderId, ...
Preconditions:   - The consumer exists and can place orders.
                 - The line items correspond to the restaurant's menu items.
```

```
                - The delivery address and time can be serviced by the restaurant.
Post-conditions: - The consumer's credit card was authorized for the order total.
                - An order was created in the PENDING_ACCEPTANCE state.
```

Türkçesiyle: ön koşullar, tüketicinin var olması ve sipariş verebilmesi, sipariş kalemlerinin restoranın menüsündeki ürünlere karşılık gelmesi, teslim adresi ve zamanının restoran tarafından karşılanabilmesidir. Son koşullar, tüketicinin kredi kartının sipariş toplamı için onaylanmış olması ve siparişin PENDING_ACCEPTANCE (kabul bekliyor) durumunda oluşturulmuş olmasıdır. Sunucu bunun derste görülen işlem belirtimiyle aynı biçimde olduğunu belirtir: işlem, dönüş değerleri, ön koşul, son koşul (01:12:39-01:12:48).

17. İş yeteneklerine göre ayrıştırma

Slayttaki içerik (ekrandan, 01:15:50): bir organizasyonun iş yetenekleri (business capabilities), organizasyonun **amacı**, **yapısı** ve **iş süreçleri** analiz edilerek belirlenir. Her iş yeteneği bir servis gibi düşünülebilir; farkı, **teknik değil iş odaklı** olmasıdır. Belirtimi girdiler, çıktılar ve hizmet düzeyi anlaşmaları (SLA) gibi bileşenlerden oluşur.

Kitaptan alıntı: üst düzey yetenekler Tedarikçi yönetimi, Tüketici yönetimi, Sipariş alma ve karşılama ve Muhasebedir. Pazarlamayla ilgili başka üst düzey yetenekler de olabilir. Çoğu üst düzey yetenek alt yeteneklere ayrıştırılır; örneğin Sipariş alma ve karşılama beş alt yeteneğe ayrılır.

Slayttaki örnek iş yetenekleri:

- Supplier management (tedarikçi yönetimi)
  - Courier management: kurye bilgilerini yönetmek.
  - Restaurant information management: restoran menülerini ve konum, açık saatler gibi diğer bilgileri yönetmek.
- Consumer management (tüketici yönetimi): tüketicilerle ilgili bilgileri yönetmek.
- Order taking and fulfillment (sipariş alma ve karşılama)
  - Order management: tüketicilerin sipariş oluşturmasını ve yönetmesini sağlamak.
  - Restaurant order management: siparişlerin restoranda hazırlanmasını yönetmek.
  - Logistics (lojistik).
  - Courier availability management: kuryelerin siparişleri teslim etmeye anlık uygunluğunu yönetmek.
  - Delivery management: siparişleri tüketicilere teslim etmek.
- Accounting (muhasebe)
  - Consumer accounting: tüketicilerin faturalandırılması.
  - Restaurant accounting: restoranlara yapılan ödemeler.
  - Courier accounting: kuryelere yapılan ödemeler.

Sunucunun anlatımı (01:13:00-01:14:28): sistem belli iş gereksinimlerine göre düzenlenir. Bu yapının sürekli geliştirilebilir, kararlı olması, ileride görülecek ortak kapanış ilkesine uyması ve parçalar arasında olabildiğince az ilişki olması gerekir. Jeff Bezos'un "iki pizza takımı" (two-pizza team) dediği bir yapı vardır: bir takımın fazla kalabalık olup olmadığını anlamak için takım iki pizzayla doyuyorsa normaldir; iki pizzadan fazlası gerekiyorsa takım büyümüştür. Her takım, bir servisi ya da bir grup servisi diğerlerinden ayrı olarak yönetebilmelidir.

İş yetenekleri servislere çevrilirken gruplamalar yapılır (01:16:10-01:17:56). Önce iş tarafında nelere ihtiyaç olduğu bilinmelidir. Yetenekler en küçük birimlerine kadar alt kısımlara bölünür, sonra birbirine benzeyenler toplanır. Örneğin kurye ve teslimat ayrı yeteneklerdir ama ikisi de lojistik altında toplanabilir; bu durumda tek bir serviste birleştirilir. Karmaşıklığı artırmadan birleştirilebilenler tek servis olur; yine de farklı servislere ihtiyaç duyanlar farklı servislerden hizmet verir. Bu servislere bölme yapısı, ileride API'lerin nasıl ayrılacağına da örnek olur.

18. Alt alanlara göre ayrıştırma ve hocanın ara özeti

Alt alan kalıbı (subdomain pattern) alan güdümlü tasarımdan (Domain-Driven Design) gelir. Problem parçalara bölünür ve bu parçalar tek tek yönetilerek sonuca ulaşılır. Önce problem en küçük parçasına kadar bölünür, sonra parçalar ait oldukları alana (domain) göre yeniden gruplanır (01:17:56-01:18:12).

Hoca burada bir ara özet yapar (01:18:50): buraya kadarki konu servislerin nasıl bölüneceği ve nasıl keşfedileceğidir. Önce problem anlaşılır ve modellenir. Sonra bu model, "bunu işlevsel parçalara nasıl bölerim?" diye eşlenir. Ardından bu işlevsel parçalardan servisler çıkarılır: bazıları daha fazla bölünmelidir, bazıları doğrudan servis olarak modellenebilir, bazıları birbirine o kadar yakın işlerdir ki tek bir servis iki üç işi birden yapabilir. Amaç her servisin tekil bir sorumluluk noktası olmasıdır; hoca bunu "yumurtayı parçalamak, kabuğunu kırmak" diye anlatır. Yapılmaya çalışılan şey, alanı alt alanlara bölüp her birini belli servislere eşlemektir.

19. Ayrıştırma ilkeleri: SRP ve CCP

Kitap ayrıştırma ilkelerini iki başlıkta inceler (01:19:52).

19.1 Tek sorumluluk ilkesi (Single Responsibility Principle)

Slayttaki içerik (ekrandan, 01:20:00):

- Yazılım mimarisi ve tasarımının ana hedeflerinden biri her yazılım öğesinin sorumluluklarını belirlemektir.
- Tek sorumluluk ilkesi: **"Bir sınıfın değişmek için yalnız bir nedeni olmalıdır."** (Robert C. Martin)
- SRP mikroservis mimarisi tanımlanırken uygulanabilir: her birinin **tek sorumluluğu** olan küçük, uyumlu (cohesive) servisler oluşturulur.
- Bu, **servislerin boyutunu küçültür** ve **kararlılıklarını artırır**.

Sunucu bunu Unix felsefesine benzetir: Unix'te her program tek bir işe odaklanmalı, ama o işi çok iyi yapmalıdır (01:20:00). Sistem en küçük ve anlamlı parçalardan oluştuğu için test kolaylaşır, anlaşılabilirlik artar ve sistem daha kararlı olur.

19.2 Ortak kapanış ilkesi (Common Closure Principle)

Slayttaki içerik (ekrandan, 01:21:15):

- **"Bir paketteki sınıflar aynı tür değişikliklere karşı birlikte kapalı olmalıdır. Bir paketi etkileyen değişiklik o paketteki bütün sınıfları etkiler."** (Robert C. Martin)
- Fikir şudur: iki sınıf aynı temel nedenle birlikte (lockstep) değişiyorsa **aynı pakete aittir**.
- Amaç, bir iş kuralı değiştiğinde geliştiricilerin **az sayıda pakette (ideal olarak yalnız birinde)** kod değiştirmesidir.
- CCP'yi uygulamak, bir gereksinim değiştiğinde değiştirilip dağıtılması gereken servis sayısını **en aza indirir**. İdeal olarak bir değişiklik yalnız tek bir takımı ve tek bir servisi etkiler.
- CCP, dağıtık monolit **anti-kalıbının panzehiridir**.

Bu ilke, hocanın 14. bölümdeki düzeltmesiyle doğrudan bağlantılıdır: bir değişiklik birden çok servisi birlikte değiştirmeyi gerektiriyorsa CCP ihlal edilmiştir ve ortaya dağıtık monolit çıkar.

20. Ayrıştırmanın engelleri

Sunumda dört engel anılır: ağ, senkronizasyon, veri tutarlılığı ve iç içe geçmiş, birbirine bağlı yapı (01:21:55).

20.1 Ağ gecikmesi ve toplu (batch) işlemler

Servisler genelde uzak sunucularda dağıtık çalışır; yakın sunucularda bile aralarında fiziksel bir ağ vardır. Servisler arası gidip gelme ağı yorar. Kitaba göre bu sorun toplu (batch) işlemlerle çözülebilir (01:22:11-01:22:37).

Sunucu toplu işlemi veritabanı üzerinden anlatır (01:22:48): her bir insert işlemi 80-100 milisaniye sürsün ve gereksinim 10 işlemin 500 milisaniyede yapılması olsun. İşlemler tek tek yapılırsa süre işlem sayısıyla doğrusal artar; istek sayısı 20'ye çıktığında gereksinimin kat kat üstüne çıkılır. Toplu işlem ağ trafiğini azalttığı için, normalde bir insert'i 90 milisaniyede yapan bir sistem 100, 200, hatta 1000 kaydı bazen 100-120 milisaniyede ekleyebilir.

> **Ders sonrası not:** Sunucu "5 işlem için 9 kere 5, 450 milisaniye" der; burada büyük olasılıkla işlem başına 90 ms kastedilmektedir: $5 \times 90 = 450$ ms. Aynı varsayımla 10 işlem tek tek yapılırsa $10 \times 80 = 800$ ile $10 \times 100 = 1000$ ms arası sürer, yani 500 ms gereksinimi karşılanamaz (python ile doğrulandı).

Sunucu bu yapıyı birkaç yerde gördüğünü söyler: servisler arka planda asenkron çalışır, gelen istekleri kullanıcıyı bekletmeyecek kısa sürelerde biriktirip toplu işlemle yazar; böylece veritabanı trafiği de azalır (01:23:53-01:24:21).

20.2 Senkron iletişim

Servisler arası haberleşme olabildiğince az olmalıdır; az olmazsa servisler birbirini beklemek zorunda kalır (01:24:21). Web uygulamalarında bir servis sunucuya istek attığında cevap gelene kadar ekran kilitlenebilir ve başka işlem yapılamaz; mobil uygulamalarda da benzer sorunlar olur. Bir servisin beklemesi bütün sistemin kilitlenmesine yol açabilir. Bu yüzden servisler asenkron çalışacak biçimde yazılır ve birbirinden olabildiğince yalıtılırsa sistem daha tutarlı hâle gelir (01:24:45-01:25:04).

20.3 Veri tutarlılığı

Veri tutarlılığı daha kritik bir problemdir (01:25:21). Bir işlem, veritabanındaki bir yapıyı iki farklı servisin değiştirmesini gerektirebilir. Verinin tutarlı kalması için aralarında senkronizasyon gerekir. A servisi işlem yaparken B servisi beklerse sistem yavaşlar. Beklemezse sorun çıkar: sunucunun örneğinde 10 liralık bir fatura üzerinde A servisi bir indirim, B servisi başka bir indirim hesaplar; ikisi de birbirinden habersiz sonuçlarını yazarsa biri diğerinin yazdığını ezer ve veri tutarsız olur (01:26:00-01:26:45).

Sunucuya göre çözüm atomik değişkenler ya da atomik yapılar kullanmaktır; örneğin Oracle veritabanında bir işlem yapılırken ilgili alanın kilitlenmesi gibi (01:26:45).

> **Ek bilgi (derste anlatılmadı):** Tek veritabanında kilit ve ACID transaction ile çözülen bu sorun, her servisin kendi veritabanı olan mikroservislerde dağıtık bir transaction gerektirir. Richardson'ın kitabı bunun için büyük olasılıkla saga kalıbını önerir: bir dizi yerel transaction ve hata olursa önceki adımları geri alan telafi (compensating) transaction'ları. Hocanın sonraki derste anlatılacağını söylediği konu da budur.

20.4 Tanrı sınıfları (god classes)

Slayttaki içerik (ekrandan, 01:27:30):

- Ayrıştırmanın bir başka engeli **tanrı sınıfları** denen sınıflardır. Tanrı sınıfları, uygulamanın her yerinde kullanılan şişkin sınıflardır.
- Bir tanrı sınıfı genellikle uygulamanın pek çok farklı yönü için iş mantığını gerçekler. Normalde çok sütunlu bir veritabanı tablosuna eşlenen çok sayıda alanı vardır.
- Çoğu uygulamada alanın merkezindeki kavramı temsil eden en az bir tanrı sınıfı vardır: bankacılıkta hesaplar, e-ticarette siparişler, sigortacılıkta poliçeler gibi.
- Tanrı sınıfı uygulamanın pek çok farklı yönünün durumunu ve davranışını bir arada topladığı için, onu kullanan iş mantığını servislere bölmenin önünde aşılmaz bir engeldir.
- Örnek: Order sınıfı bir tanrı sınıfıdır.

Slayttaki sınıf diyagramı metne dökülürse:

```
Order
  orderTotal, deliveryTime, status
  <<delivery>>     pickupTime
  <<billing>>      transactionId
  <<orderTaking>>  create(), cancel()
  <<restaurant>>   accept(), reject(), noteReadyForPickup()
  <<delivery>>     assignCourier(), notePickedUp(), noteDelivered()

Order → Address, Courier, Consumer, Restaurant, PaymentInfo
Order → OrderLineItem
```

Sunucunun anlatımı (01:27:13-01:28:15): Order sınıfı sistemin içinde her şeyin ondan beslendiği bir tanrı sınıfı hâline gelir. Bu monolitik yapıda belki sorun olmaz ama mikroservis yapısında sıkıntı çıkarır. Çözüm olarak aralarındaki ilişkiler tanımlanıp işlevsellik yerine özellik olarak atanır. Örneğin Order sınıfının kendi içinde adres oluşturması olmamalıdır. Bu tür sorunlar yazılım mimarileri ve tasarım kalıplarıyla aşılabilir.

> **Ek bilgi (derste anlatılmadı):** Richardson'ın kitabında önerilen çözüm büyük olasılıkla, alan güdümlü tasarımdaki gibi her servise kendi Order modelini vermektir: Order Service'te siparişin kendisi, Kitchen Service'te bir Ticket, Delivery Service'te bir Delivery nesnesi bulunur ve her biri yalnız kendi alt alanının alanlarını ve davranışını taşır.

21. Sistem operasyonlarını servislere atamak ve API tasarımı

Son konu servis API'lerinin tanımlanması ve servislerin birbiriyle haberleşmesidir (01:28:37). En küçük parçalara bölünüp sonra gruplanarak işlevsel bir yapı elde edilmişti; bu yapının parçaları birbirini olabildiğince az çelişecek biçimde kullanılarak sonuca varılır (01:29:06).

Slayttaki içerik (ekrandan, 01:30:00): ilk adım, bir istek için hangi servisin ilk giriş noktası olacağına karar vermektir. Birçok sistem operasyonu bir servise düzgünce eşlenir, ama bazen eşleme o kadar açık değildir. Örneğin kuryenin konumunu güncelleyen noteUpdatedLocation() işlemi: bir yandan kuryelerle ilgili olduğu için Courier servisine atanmalıdır; öte yandan kurye konumuna ihtiyaç duyan Delivery Service'tir. Bu durumda işlemi, işlemin sağladığı bilgiye ihtiyaç duyan servise atamak daha iyi bir seçimdir. Başka durumlarda işlemi, onu karşılamak için gereken bilgiye sahip servise atamak mantıklı olabilir.

Slayttaki eşleme tablosu:

Servis	İşlemler
Consumer Service	`createConsumer()`
Order Service	`createOrder()`
Restaurant Service	`findAvailableRestaurants()`
Kitchen Service	`acceptOrder()`, `noteOrderReadyForPickup()`
Delivery Service	`noteUpdatedLocation()`, `noteDeliveryPickedUp()`, `noteDeliveryDelivered()`

Sunucu, servislerin diğerlerine belli bağlantılarının olacağını ve API tasarlanırken bunların da ele alınması gerektiğini söyler; kitabın 4., 6., 7. ve 8. bölümlerine atıf yapıldığını belirtir (01:29:56-01:30:13). Hoca bunların sonraki bölümlerde ayrıntılı anlatılacağını, bu bölümün sonundaki kısmın yalnız bir tanıtım olduğunu ekler (01:31:11).

22. Hocanın kapanış özeti

Hoca ikinci sunumu şöyle özetler (01:31:26): servisler nasıl bulunur, nasıl ayrıştırılır ve bununla ilgili strateji nasıl olmalıdır? Derste birlikte görüldüğü gibi önce bir analiz modeli çıkarılır, sonra bu model adım adım çalışan, işlevsel olarak bir arada anlamlı parçalara getirilir ve ayrıştırılır.

Dersin Toparlaması

Dersin ilk sunumu monolitik mimarinin neden bir "cehennem"e dönüşebildiğini gösterdi: tek kod tabanı ve tek dağıtım birimi küçük uygulamalarda geliştirmeyi, testi, dağıtımı ve ölçeklemeyi kolaylaştırır; ama büyüdükçe güvenilirlik (tek hata noktası), her değişiklikte tam yeniden dağıtım ve teknoloji yığınına bağlılık sorunları ortaya çıkar. Mikroservis mimarisi uygulamayı bağımsız dağıtılabilen servislere bölerek sürekli teslim, özerk takımlar ve hata yalıtımı sağlar; bedeli dağıtık sistem karmaşıklığı ve hocanın vurguladığı orkestrasyon problemleridir. Bu problemler için mikroservis kalıp dili ve DevOps uygulamaları kullanılır.

İkinci sunum ayrıştırmanın nasıl yapılacağını anlattı: 4+1 görünüm modeli ve katmanlı ile altıgen mimari stilleri, servis API'sinin komut, sorgu ve olaylardan oluşması, gevşek bağlılık, üç adımlı tanımlama süreci (sistem operasyonları, servisler, API'ler ve iş birlikleri), iş yeteneklerine ve alt alanlara göre ayrıştırma, SRP ve CCP ilkeleri ile ağ gecikmesi, senkron iletişim, veri tutarlılığı ve tanrı sınıfları engelleri. Hocanın en önemli düzeltmesi, servis boyutunun değil, bir değişikliğin tek servisle sınırlı kalmasının önemli olduğudur; aksi hâlde dağıtık bir monolit ortaya çıkar. Hocanın söylediğine göre sonraki derste kitabın 3. bölümüyle mikroservis mimarisinde süreçler arası iletişim ve transaction'ları yönetmek için saga kalıbının mikroservislerde nasıl uygulandığı işlenecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.